The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch18_generative

AYuSh

Contents

Chapter 18: Generative Models 3

Chapter 18: Generative Models

Discriminative models answer questions about data; generative models make more of it. The interview subject is really four competing answers to one problem — *learn a distribution you can sample from* — each buying a different corner of an impossible triangle: sample quality, sampling speed, and tractable likelihood. VAEs optimize a likelihood *bound* and sample fast but blur; GANs produce sharp samples with no likelihood at all and famously unstable training; normalizing flows give *exact* likelihoods at the price of architectural handcuffs; diffusion models took the quality crown by spending compute at sampling time. Every section here is a member of that trade-space, and the closing section builds the referees (FID, Inception Score) that judge them.

The measurements: a plain autoencoder whose latent space works for reconstruction and fails as a prior — 34% of $\mathcal{N}(0, I)$ samples land in holes no training point occupies (Listing 1); a VAE with the ELBO derived and every gradient — including the path *through the sampled latent* — checked at 5×10^{-9}, whose aggregate posterior sits at mean ≈ 0, std ≈ 0.9 and whose prior samples decode to digits (Listing 2); the reparameterization trick justified by measurement — pathwise gradient variance 4.0 vs REINFORCE's 220 on the same objective (Listing 3); a GAN on the 8-Gaussian ring where the original minimax loss covers 0 of 8 modes (its gradient measured 50× too weak exactly when the discriminator wins) and the non-saturating fix covers 7 unevenly — mode collapse on camera (Listing 4); a DDPM built from the closed-form forward corruption and an ε-predicting network, recovering two-moons from pure noise (Listing 5); classifier-free guidance swept from $w = 0$ to $w = 4$, with class purity rising 0.80 → 1.00 while on-manifold fidelity peaks and then *falls* — the guidance trade-off as a table (Listing 6); a RealNVP flow inverted to 10^{-15} *before training* (invertible by construction) and trained by exact maximum likelihood, 2.80 → 1.87 nats (Listing 7); and FID/IS implemented from scratch, with mode collapse scoring FID 28.7 / IS 1.13 and each metric's blind spot demonstrated (Listing 8).

Autoencoders: compression is not generation

An autoencoder learns $x \to z \to \hat{x}$ through a bottleneck, trained on reconstruction alone. It is a fine representation learner (Chapter 8 used one for anomaly detection) — and a broken generator, for a reason worth stating precisely: *nothing constrains where the encoder puts things*. Listing 1's 2-D latent space carries digit clusters at mean $(-2.6, -0.1)$, arbitrary scale, with voids between clusters; sample $z \sim \mathcal{N}(0, I)$ as if it were a prior and 34% of draws land more than a unit from *any* encoded training point — the decoder, asked to decode coordinates it has never seen, produces the smeared non-digit in the figure. Generation needs the latent space to be *covered by a known distribution*, which is exactly the constraint the VAE adds.

VAEs: the ELBO and the reparameterization trick

The VAE (Kingma & Welling, 2013) makes the latent probabilistic: the encoder outputs a *distribution* $q_\phi(z|x) = \mathcal{N}(\mu(x), \sigma^2(x))$, the decoder a likelihood $p_\theta(x|z)$, and the prior is fixed at $\mathcal{N}(0, I)$. The interview derivation, in three lines: $\log p(x)$ is intractable (the integral over z); insert q and apply Jensen's inequality to get the **evidence lower bound**

$$\log p(x) \geq \mathbb{E}_{q(z|x)}[\log p(x|z)] - \mathrm{KL}(q(z|x) \,\|\, p(z))$$

— reconstruction minus a regularizer pinning each posterior to the prior, with the gap between bound and truth equal to $\mathrm{KL}(q \| p(z|x))$, so a better posterior approximation *is* a tighter bound. For Gaussian q and prior, the KL has the closed form $\frac{1}{2}\sum(\sigma^2 + \mu^2 - 1 - \log\sigma^2)$ — Listing 2 implements it, and its gradients, by hand.

The trick that makes any of this trainable: the reconstruction term is an expectation over a *sampled* z, and "backprop through a sample" is undefined — unless you rewrite the sample as $z = \mu + \sigma \odot \varepsilon$, $\varepsilon \sim \mathcal{N}(0, I)$: the randomness moves outside the computation graph, and z becomes a deterministic, differentiable function of μ, σ. Listing 2's gradient check *freezes* ε and differentiates straight through the sample: 5.2×10^{-9}. Trained, the aggregate posterior lands at mean $(0.08, -0.21)$, std $(0.92, 0.80)$ — compare the AE's $(-2.6, \cdot)$ — and prior samples decode to recognizable digits (the figure's panel). The costs to volunteer: VAE samples are characteristically blurry (a pixel-wise Gaussian/Bernoulli likelihood makes the decoder average over posterior uncertainty), and **posterior collapse** — if the decoder is strong enough to ignore z, the KL term happily drives $q \to p$ and the latent dies; remedies include KL annealing, free bits, and weaker decoders. The β-VAE knob (scale the KL) trades reconstruction for disentanglement in the same equation.

Why reparameterization: the variance argument

The alternative gradient-through-sampling estimator is the **score function / REINFORCE** identity, $\nabla_\mu \mathbb{E}[f(z)] = \mathbb{E}[f(z)\nabla_\mu \log p(z;\mu)]$ — no differentiability of f needed, works for discrete z, and is the policy-gradient backbone of Chapter 32. Listing 3 puts both estimators on the same toy objective ($\mathbb{E}_{z\sim\mathcal{N}(\mu,1)}[z^2]$, true gradient 2μ): both are unbiased, but the pathwise estimator's variance is **4.0 against REINFORCE's 220** — 55× — and a mean baseline only cuts REINFORCE to 81. The rule it teaches: when f is differentiable and z reparameterizable (Gaussians are), pathwise wins outright; REINFORCE is what remains for discrete latents (or Gumbel-softmax relaxations) and non-differentiable rewards. That single variance table is why the VAE works and why RL gradients are noisy — same estimator, opposite circumstances.

GANs: the adversarial game and its pathologies

The GAN (Goodfellow et al., 2014) abandons likelihood entirely: a generator G maps noise to samples, a discriminator D classifies real vs fake, trained as a minimax game $\min_G \max_D \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$. At the (theoretical) optimum $D^* = p_{data}/(p_{data} + p_g)$ and G's objective reduces to minimizing the Jensen-Shannon divergence. Two famous pathologies, both *measured* in Listing 4. **Saturation**: early in training D wins easily, $D(G(z)) \approx 0$, and the minimax generator gradient $\propto 1/(1 - D)$ is ≈ 1 — flat exactly when learning is most needed; the **non-saturating** loss $-\log D(G(z))$ has gradient $\propto 1/D \approx 50$ at the same point (the listing's table: 50× at $D(fake) = 0.02$). On the 8-Gaussian ring the difference is total: minimax covers **0/8 modes**; non-saturating covers 7/8 — with per-mode counts 2 to 74, which is the second pathology, **mode collapse**: the generator concentrates on modes that currently fool D, and nothing in the objective pays for coverage (contrast maximum likelihood, which pays infinitely for assigning zero mass to real data). The stabilization toolkit interviews expect by name: Wasserstein GAN (critic + weight clipping) and WGAN-GP (gradient penalty) replacing JS with an Earth-Mover objective whose gradients don't saturate; spectral normalization constraining D's Lipschitz constant; two-timescale learning rates; minibatch discrimination against collapse. Architecture lineage in one breath: DCGAN (all-convolutional, BN, the baseline recipe), StyleGAN (mapping network to a *style* space $\mathcal{W}$, per-layer style injection, noise inputs — the disentanglement standard), and the honest 2020s footnote: diffusion took the quality crown, GANs kept the speed one (single forward pass, real-time).

Diffusion models

DDPM (Ho et al., 2020) generates by *learning to reverse gradual noising*. Forward process: T steps of $q(x_t|x_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}\, x_{t-1}, \beta_t I)$ — no parameters, and with $\bar{\alpha}_t = \prod(1 - \beta_s)$ it telescopes to the closed form $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon$: any corruption level in one jump, which is what makes training cheap (sample t uniformly, corrupt, learn). Listing 5's schedule drives $\bar{\alpha}_T$ to 0.0096 — signal gone, $x_T \approx \mathcal{N}(0, I)$. The training objective — the ELBO of this hierarchical model, simplified by Ho et al. — collapses to something absurdly clean: **predict the noise**, $\min \|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$. The network is a denoiser conditioned on t (sinusoidal time embedding, same trick as Chapter 16's positions). Sampling walks backward: $x_{t-1} = \frac{1}{\sqrt{\alpha_t}}(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta) + \sqrt{\beta_t}\, z$ — Listing 5 implements exactly this and pulls two-moons out of pure noise (median distance to manifold 0.068; the figure shows the forward smearing and the reverse samples). The connections worth naming: ε-prediction is equivalent to **score matching** ($\varepsilon_\theta \propto -\nabla \log p(x_t)$, the score), sampling is discretized Langevin dynamics / an SDE, and DDIM makes the reverse deterministic and skippable (50 steps instead of 1000) — speed being diffusion's tax, one network pass *per step* against the GAN's single pass. **Latent diffusion / Stable Diffusion**: run the whole machine inside a VAE's latent space (8×

downsampled — ~64× cheaper per step), condition on CLIP-style text embeddings via cross-attention (Chapters 16-17), and that is the modern text-to-image stack in one sentence.

Guidance

Conditional diffusion learns $\varepsilon_\theta(x_t, t, c)$; **classifier-free guidance** (Ho & Salimans, 2021) makes conditioning *steerable* with one trick: during training, drop the condition to a null token 10% of the time, so one network learns both $\varepsilon(x|c)$ and $\varepsilon(x)$; at sampling, extrapolate *away* from unconditional: $\hat{\varepsilon} = (1+w)\,\varepsilon(x|c) - w\,\varepsilon(x)$. Listing 6 trains it on labeled two-moons and sweeps w: class purity 0.799 → 0.939 → 1.000 at $w = 0, 1, 4$, while on-manifold fidelity rises to 0.704 at $w = 1$ then *collapses* to 0.361 at $w = 4$ and sample spread inflates — over-guidance pushes samples off the data manifold, the 2-D miniature of the over-saturated, fried look of high-guidance images. That fidelity-diversity dial (and its typical sweet spot around 5-10 in real text-to-image systems) is a current-events interview staple.

Normalizing flows

Flows are the likelihood purist's answer: an *invertible* network f with the change-of-variables formula $\log p(x) = \log p_z(f(x)) + \log|\det J_f(x)|$ — exact density, exact sampling via f^{-1}, no bounds, no adversaries. The whole game is making $\det J$ cheap: **affine coupling** (RealNVP) splits dimensions, passes half through untouched, and affine-transforms the other half with scale/shift networks *of the passed half* — the Jacobian is triangular, its log-det just $\sum s$, and inversion is algebraic (subtract, divide) *regardless of how complicated the s,t networks are* — they are never inverted. Listing 7 stacks six couplings with alternating masks and demonstrates the punchline before any training: inverse(forward(x)) agrees to 1.3×10^{-15} — invertibility is architectural, not learned. Trained by exact maximum likelihood (NLL 2.80 → 1.87 nats), the flow warps two-moons into a Gaussian and back — the figure's three panels are the definition of a flow in pictures. Costs: dimension-preserving (latent = data dimension, no compression), architectural handcuffs (invertibility limits expressiveness; hence deep stacks, 1×1 convolutions in Glow), and historically weaker sample quality — which is why flows live on in density estimation, anomaly detection, and as components (VAE priors, diffusion's noise schedules) rather than as flagship generators.

Evaluating generators: IS and FID

Generative evaluation is its own interview question because the obvious answer — likelihood — is unavailable (GANs), only bounded (VAEs), or uncorrelated with visual quality (high-likelihood models can produce garbage samples and vice versa). **Inception Score** feeds samples through a fixed classifier: sharp per-sample predictions (confident

$p(y|x)$) and diverse marginal ($p(y)$ near uniform) multiply into $\exp(\mathbb{E}[\mathrm{KL}(p(y|x)\|p(y))])$ — ceiling = number of classes. Its blind spot: *it never looks at real data* — a generator memorizing one perfect image per class scores near-ceiling. **FID** fixes that by comparing feature *distributions*: fit Gaussians to real and generated features from the classifier's penultimate layer, then the Fréchet distance $\|\mu_1-\mu_2\|^2+\mathrm{Tr}(C_1+C_2-2(C_1C_2)^{1/2})$. Listing 8 builds both on digits with a 97.3%-accurate stand-in Inception and stages the lineup: identical sets score FID 0.00 (sanity), a fresh real sample 0.31 (the sampling-noise floor — FID is biased at small n; compare at fixed sample count), noise/blur degrade in order, and **mode collapse is the star exhibit**: all-ones scores FID 28.7 (means fine, *covariances* collapsed — FID's second term is why it catches diversity failures) and IS 1.13. Remaining caveats for the well-prepared: FID inherits its feature extractor's biases (ImageNet features judge texture more than structure), says nothing about memorization (nearest-neighbor checks do), and precision/recall-for-distributions decomposes quality vs coverage when one number won't do.

Code implementations

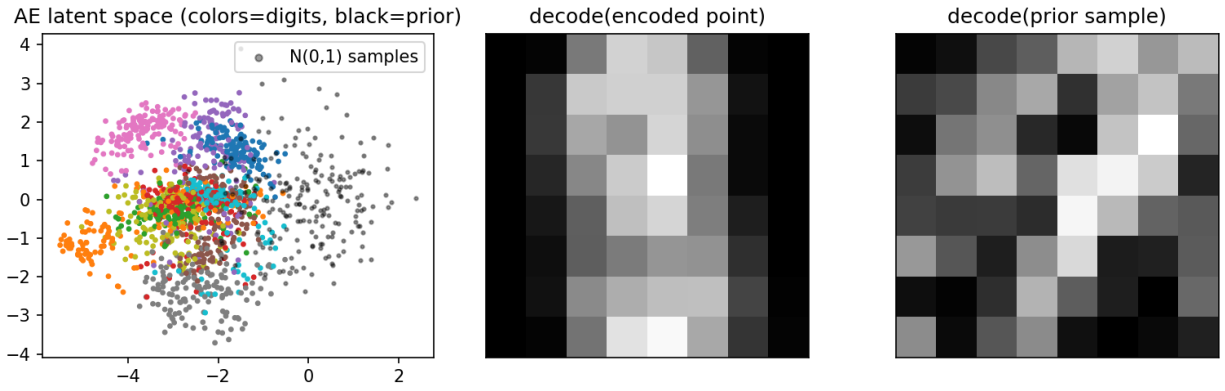
Listing 1 — A plain autoencoder: great reconstruction, unusable prior

```
"""Listing 1: a plain autoencoder -- great reconstruction, unusable prior."""
import numpy as np
from sklearn.datasets import load_digits
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
rng = np.random.default_rng(0)

X, y = load_digits(return_X_y=True); X = X/16.0
idx = rng.permutation(len(X)); tr, te = idx[:1400], idx[1400:]
D, Hd, Z = 64, 64, 2

r = np.random.default_rng(1)
W1 = r.normal(0,(2/D)**.5,(D,Hd)); b1 = np.zeros(Hd)
W2 = r.normal(0,(2/Hd)**.5,(Hd,Z)); b2 = np.zeros(Z)
W3 = r.normal(0,(2/Z)**.5,(Z,Hd)); b3 = np.zeros(Hd)
W4 = r.normal(0,(2/Hd)**.5,(Hd,D)); b4 = np.zeros(D)

def enc(x):
    h = np.maximum(0, x@W1+b1); return h, h@W2+b2
def dec(z):
    h = np.maximum(0, z@W3+b3); return h, 1/(1+np.exp(-(h@W4+b4)))

lr = 0.02
for it in range(40000):
    x = X[tr[rng.integers(len(tr))]]
    h1, z = enc(x); h2, xh = dec(z)
    d4 = (xh - x)/D                                   # BCE-at-logits grad
    dW4 = np.outer(h2, d4); dh2 = (W4@d4)*(h2 > 0)
    dW3 = np.outer(z, dh2); dz = W3@dh2
    dW2 = np.outer(h1, dz); dh1 = (W2@dz)*(h1 > 0)
    dW1 = np.outer(x, dh1)
    for P, G in [(W4,dW4),(b4,d4),(W3,dW3),(b3,dh2),(W2,dW2),(b2,dz),(W1,dW1),
(b1,dh1)]:
        P -= lr*G
rec = np.mean([( (dec(enc(X[i])[1])[1] - X[i])**2 ).mean() for i in te])
print(f"AE test reconstruction MSE: {rec:.4f}")
```

```
Ztr = np.array([enc(X[i])[1] for i in tr])
print(f"latent stats: mean {np.round(Ztr.mean(0),2)}  std {np.round(Ztr.std(0),2)}
(nothing ties this to N(0,1))")

# sample "from the prior": z ~ N(0, I) mostly lands OUTSIDE the data's latent region
zs = rng.normal(size=(1000, Z))
from scipy.spatial import cKDTree
d_near = cKDTree(Ztr).query(zs)[0]
print(f"fraction of N(0,1) samples farther than 1.0 from ANY encoded training point:
{(d_near>1).mean():.2f}")

fig, axes = plt.subplots(1, 3, figsize=(10.5, 3.4))
sc = axes[0].scatter(Ztr[:,0], Ztr[:,1], c=y[tr], cmap='tab10', s=5)
axes[0].scatter(zs[:200,0], zs[:200,1], c='k', s=4, alpha=.4, label='N(0,1) samples')
axes[0].set_title('AE latent space (colors=digits, black=prior)');
axes[0].legend(markerscale=2)
for ax, z0, t in [(axes[1], Ztr[3], 'decode(encoded point)'), (axes[2],
rng.normal(size=Z)*1.5, 'decode(prior sample)')]:
    ax.imshow(dec(z0)[1].reshape(8,8), cmap='gray'); ax.set_title(t);
ax.set_xticks([]); ax.set_yticks([])
fig.tight_layout(); fig.savefig("figures/ch18/fig_l1_ae.png", dpi=150)
print("figure saved: fig_l1_ae.png")
```

Output:

```
AE test reconstruction MSE: 0.0552
latent stats: mean [-2.62 -0.09]  std [0.96 1.29]  (nothing ties this to N(0,1))
fraction of N(0,1) samples farther than 1.0 from ANY encoded training point: 0.34
figure saved: fig_l1_ae.png
```



Listing 2 — A VAE from scratch: ELBO, reparameterization, and a prior you can sample

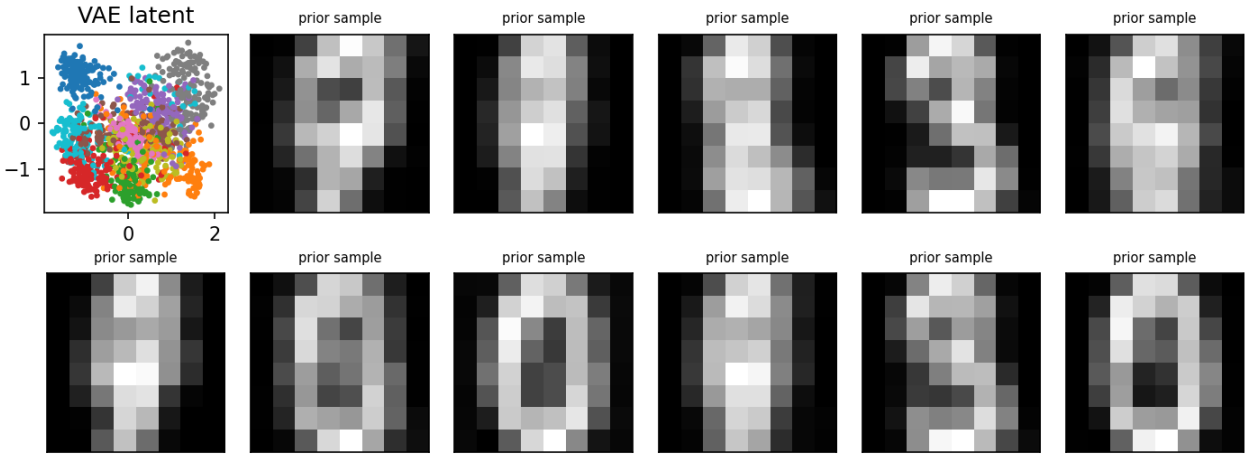
```
"""Listing 2: a VAE from scratch -- ELBO, reparameterization, and a prior you can
actually sample."""
import numpy as np
from sklearn.datasets import load_digits
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
rng = np.random.default_rng(0)

X, y = load_digits(return_X_y=True); X = X/16.0
idx = rng.permutation(len(X)); tr, te = idx[:1400], idx[1400:]
D, Hd, Z = 64, 64, 2

r = np.random.default_rng(1)
```

```
P = dict(W1=r.normal(0,(2/D)**.5,(D,Hd)), b1=np.zeros(Hd),
         Wm=r.normal(0,(1/Hd)**.5,(Hd,Z)), bm=np.zeros(Z),
         Wv=r.normal(0,(1/Hd)**.5,(Hd,Z)), bv=np.zeros(Z),   # predicts LOG-variance
         W3=r.normal(0,(2/Z)**.5,(Z,Hd)),  b3=np.zeros(Hd),
         W4=r.normal(0,(2/Hd)**.5,(Hd,D)), b4=np.zeros(D))

def step(x, eps, beta=1.0, update=None):
    h1 = np.maximum(0, x@P['W1']+P['b1'])
    mu = h1@P['Wm']+P['bm']; lv = h1@P['Wv']+P['bv']
    z = mu + eps*np.exp(0.5*lv)                          # REPARAMETERIZATION: noise
outside the graph
    h2 = np.maximum(0, z@P['W3']+P['b3'])
    logit = h2@P['W4']+P['b4']; xh = 1/(1+np.exp(-logit))
    bce = -(x*np.log(xh+1e-9) + (1-x)*np.log(1-xh+1e-9)).sum()
    kl = 0.5*(np.exp(lv) + mu**2 - 1 - lv).sum()
# KL( N(mu,sigma) || N(0,1) ), closed form
    L = bce + beta*kl
    # backward
    d4 = xh - x
    g = {}
    g['W4'] = np.outer(h2, d4); g['b4'] = d4
    dh2 = (P['W4']@d4)*(h2 > 0)
    g['W3'] = np.outer(z, dh2); g['b3'] = dh2
    dz = P['W3']@dh2
    dmu = dz + beta*mu                                   # recon path + KL path
    dlv = dz*eps*0.5*np.exp(0.5*lv) + beta*0.5*(np.exp(lv) - 1)
    g['Wm'] = np.outer(h1, dmu); g['bm'] = dmu
    g['Wv'] = np.outer(h1, dlv); g['bv'] = dlv
    dh1 = (P['Wm']@dmu + P['Wv']@dlv)*(h1 > 0)
    g['W1'] = np.outer(x, dh1); g['b1'] = dh1
    if update:
        for k in P: P[k] -= update*g[k]
    return L, bce, kl, g, xh

# gradient check with eps FROZEN (the reparameterized loss is deterministic given eps)
x = X[tr[0]]; eps0 = rng.normal(size=Z); _, _, _, g0, _ = step(x, eps0)
fd, worst = 1e-5, 0
for key, i2 in [('W1',(3,5)), ('Wm',(10,1)), ('Wv',(7,0)), ('W4',(20,9))]:
    W = P[key]; keep = W[i2]
    W[i2] = keep+fd; Lp,*_ = step(x, eps0); W[i2] = keep-fd; Lm,*_ = step(x, eps0);
W[i2] = keep
    num = (Lp-Lm)/(2*fd); worst = max(worst, abs(num-g0[key][i2])/max(1e-12, abs(num)
+abs(g0[key][i2])))
print(f"VAE gradient check through the reparameterized sample: worst rel err {worst:.
2e}")

for it in range(60000):
    i = tr[rng.integers(len(tr))]
    step(X[i], rng.normal(size=Z), update=3e-4)
Ls, Bs, Ks = zip(*[step(X[i], rng.normal(size=Z))[:3] for i in te])
print(f"test ELBO terms: recon(BCE) {np.mean(Bs):.1f}   KL {np.mean(Ks):.1f}   (-ELBO
{np.mean(Ls):.1f} nats)")

mus = np.array([ (np.maximum(0, X[i]@P['W1']+P['b1'])@P['Wm']+P['bm']) for i in tr])
print(f"aggregate posterior: mean {np.round(mus.mean(0),2)}  std {np.round(mus.std(0),
2)}  (pinned near N(0,1))")

def decode(z):
    h2 = np.maximum(0, z@P['W3']+P['b3']); return 1/(1+np.exp(-(h2@P['W4']+P['b4'])))
fig, axes = plt.subplots(2, 6, figsize=(9.5, 3.6))
axc = axes[0,0]; axc.scatter(mus[:,0], mus[:,1], c=y[tr], cmap='tab10', s=5);
axc.set_title('VAE latent')
zs = rng.normal(size=(11, Z))
for k, ax in enumerate(axes.ravel()[1:]):
    ax.imshow(decode(zs[k]).reshape(8,8), cmap='gray'); ax.set_xticks([]);
ax.set_yticks([])
    ax.set_title('prior sample', fontsize=7)
```

```
fig.tight_layout(); fig.savefig("figures/ch18/fig_l2_vae.png", dpi=150)
print("figure saved: fig_l2_vae.png")
```

Output:

```
VAE gradient check through the reparameterized sample: worst rel err 5.19e-09
test ELBO terms: recon(BCE) 24.5   KL 1.6   (-ELBO 26.1 nats)
aggregate posterior: mean [ 0.08 -0.21]  std [0.92 0.8 ]  (pinned near N(0,1))
figure saved: fig_l2_vae.png
```



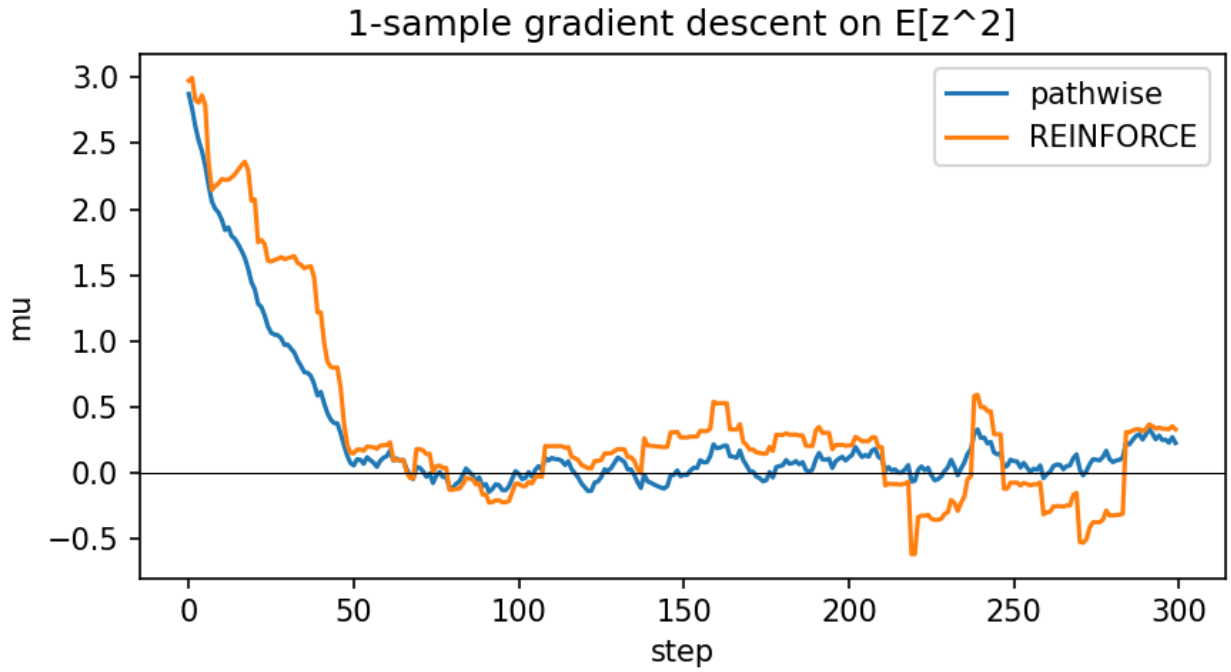
Listing 3 — Why the reparameterization trick: gradient variance, measured

```
"""Listing 3: why the reparameterization trick -- gradient variance, measured."""
import numpy as np
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
rng = np.random.default_rng(3)

# objective: minimize E_{z ~ N(mu, 1)} [ z^2 ].  True gradient d/dmu = 2*mu.
mu = 3.0
f = lambda z: z**2

N = 200000
z = mu + rng.normal(size=N)
g_path  = 2*z                                # pathwise (reparameterized): d f(mu+eps)/d
mu = f'(z)
g_score = f(z)*(z - mu)                      # score function / REINFORCE: f(z) * d log
p / d mu
b = f(z).mean()                              # variance-reduction baseline
g_base  = (f(z) - b)*(z - mu)
print(f"true gradient 2*mu = {2*mu:.1f}")
for name, g in [("pathwise (reparam)", g_path), ("REINFORCE", g_score), ("REINFORCE +
baseline", g_base)]:
    print(f"{name:22s}: mean {g.mean():7.3f}   variance {g.var():10.1f}")

# consequence: SGD with a 1-sample estimator
fig, ax = plt.subplots(figsize=(6, 3.4))
for name, kind in [("pathwise", "p"), ("REINFORCE", "s")]:
    m, traj = 3.0, []
    r2 = np.random.default_rng(0)
    for t in range(300):
        zz = m + r2.normal()
        g = 2*zz if kind == "p" else f(zz)*(zz - m)
```

```
        m -= 0.02*g; traj.append(m)
    ax.plot(traj, label=name)
    print(f"1-sample SGD, 300 steps, {name:10s}: final mu {m:7.3f} (target 0)")
ax.axhline(0, color='k', lw=.5); ax.set(xlabel='step', ylabel='mu', title='1-sample
gradient descent on E[z^2]')
ax.legend(); fig.tight_layout(); fig.savefig("figures/ch18/fig_l3_reparam.png",
dpi=150)
print("figure saved: fig_l3_reparam.png")
```

Output:

```
true gradient 2*mu = 6.0
pathwise (reparam)    : mean   6.001   variance        4.0
REINFORCE             : mean   5.993   variance      220.1
REINFORCE + baseline  : mean   5.989   variance       80.8
1-sample SGD, 300 steps, pathwise  : final mu   0.224 (target 0)
1-sample SGD, 300 steps, REINFORCE : final mu   0.328 (target 0)
figure saved: fig_l3_reparam.png
```



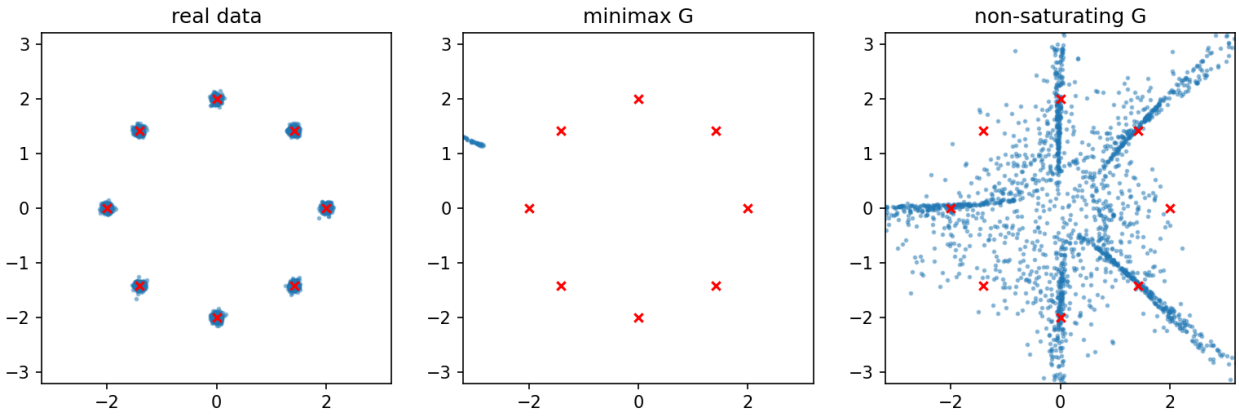
Listing 4 — A GAN on the 8-Gaussian ring: saturation and mode collapse, measured

```
"""Listing 4: a GAN from scratch on the 8-Gaussian ring -- saturation and mode
collapse, measured."""
import numpy as np
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
rng = np.random.default_rng(4)

centers = np.array([[np.cos(a), np.sin(a)] for a in np.linspace(0, 2*np.pi, 8,
endpoint=False)])*2
def real(n): return centers[rng.integers(0, 8, n)] + rng.normal(0, .05, (n, 2))
sig = lambda z: 1/(1+np.exp(-z))

def init(seed, din, h, dout):
```

```
    r = np.random.default_rng(seed)
    return [r.normal(0,(2/din)**.5,(din,h)), np.zeros(h), r.normal(0,(2/h)**.5,
(h,dout)), np.zeros(dout)]

def mlp_f(x, p):
    h = np.maximum(0, x@p[0]+p[1]); return h, h@p[2]+p[3]
def mlp_b(x, h, dout, p, lr):
    dW2 = h.T@dout; db2 = dout.sum(0)
    dh = (dout@p[2].T)*(h > 0)
    dx = dh@p[0].T
    p[2] -= lr*dW2; p[3] -= lr*db2; p[0] -= lr*(x.T@dh); p[1] -= lr*dh.sum(0)
    return dx

# 1) the saturation numbers: gradient reaching G through D early in training, two G-
losses
d_fake = np.array([0.02, 0.10, 0.50])           # D's verdict on a fake (early
training: near 0)
print("D(fake)    dL/d D  for  minimax log(1-D)   vs   non-saturating -log(D)")
for df in d_fake:
    print(f"  {df:.2f}          {1/(1-df):10.2f}                {1/df:10.2f}")
print("-> when D wins (D(fake)~0), minimax gradient ~1 vs non-saturating ~50: G learns
50x faster\n")

def train_gan(nonsat, g_steps=1, d_steps=1, iters=20000, lr=4e-3, B=64):
    G = init(0, 2, 64, 2); D = init(1, 2, 64, 1)
    for it in range(iters):
        for _ in range(d_steps):                # D: maximize log D(x) + log(1-D(G(z)))
            xr = real(B); z = rng.normal(size=(B, 2))
            hg, xf = mlp_f(z, G)
            hr, lr_ = mlp_f(xr, D); pr = sig(lr_)
            hf, lf_ = mlp_f(xf, D); pf = sig(lf_)
            mlp_b(xr, hr, (pr-1)/B, D, lr)      # real -> label 1
            mlp_b(xf, hf, (pf-0)/B, D, lr)      # fake -> label 0
        for _ in range(g_steps):
            z = rng.normal(size=(B, 2))
            hg, xf = mlp_f(z, G)
            hf, lf_ = mlp_f(xf, D); pf = sig(lf_)
            # G loss grads at D's logit: nonsat -log(D): (pf-1); minimax +log(1-D): pf
            dlogit = ((pf-1) if nonsat else pf)/B
            Dcopy = [w.copy() for w in D]
            dxf = mlp_b(xf, hf, dlogit, Dcopy, 0)
# lr=0: only need dx through frozen D
            mlp_b(z, hg, dxf, G, lr)
    xs = mlp_f(rng.normal(size=(2000, 2)), G)[1]
    dists = np.linalg.norm(xs[:, None] - centers[None], axis=2)
    covered = (dists.min(0) < 0.3)
    hit = dists.argmin(1)[dists.min(1) < 0.3]
    return xs, covered.sum(), np.bincount(hit, minlength=8)

for nonsat, tag in [(False, "minimax log(1-D)"), (True, "non-saturating -log D")]:
    xs, ncov, counts = train_gan(nonsat)
    print(f"{tag:24s}: modes covered {ncov}/8   per-mode sample counts
{counts.tolist()}")
    if nonsat: xs_ns = xs
    else: xs_mm = xs

fig, axes = plt.subplots(1, 3, figsize=(10.5, 3.4))
for ax, pts, t in [(axes[0], real(2000), 'real data'), (axes[1], xs_mm, 'minimax G'),
(axes[2], xs_ns, 'non-saturating G')]:
    ax.scatter(pts[:,0], pts[:,1], s=3, alpha=.4); ax.scatter(centers[:,0], centers[:,
1], c='r', s=25, marker='x')
    ax.set(xlim=(-3.2,3.2), ylim=(-3.2,3.2), title=t); ax.set_aspect('equal')
fig.tight_layout(); fig.savefig("figures/ch18/fig_l4_gan.png", dpi=150)
print("figure saved: fig_l4_gan.png")
```

Output:

```
D(fake)    dL/d D  for  minimax log(1-D)   vs   non-saturating -log(D)
  0.02                1.02                     50.00
  0.10                1.11                     10.00
  0.50                2.00                      2.00
-> when D wins (D(fake)~0), minimax gradient ~1 vs non-saturating ~50: G learns 50x
faster

minimax log(1-D)        : modes covered 0/8   per-mode sample counts [0, 0, 0, 0, 0, 0,
0, 0]
non-saturating -log D   : modes covered 7/8   per-mode sample counts [2, 66, 56, 0, 69,
5, 61, 74]
figure saved: fig_l4_gan.png
```



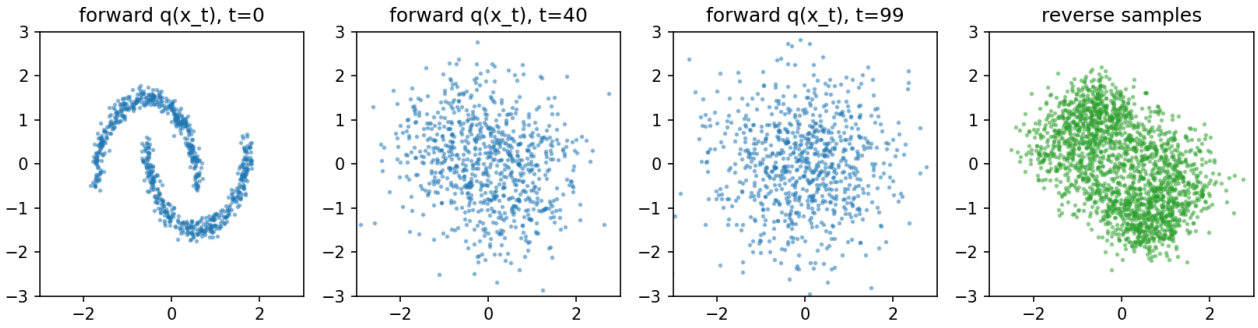
Listing 5 — DDPM from scratch: closed-form corruption, learned denoising

```python
"""Listing 5: DDPM from scratch -- forward corruption in closed form, learned reverse
denoising."""
import numpy as np
from sklearn.datasets import make_moons
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
rng = np.random.default_rng(5)

T = 100
betas = np.linspace(1e-4, 0.09, T)
alphas = 1 - betas; abar = np.cumprod(alphas)
print(f"noise schedule: abar[0]={abar[0]:.4f}  abar[T-1]={abar[-1]:.6f}  (signal all
but gone by t=T)")

def q_sample(x0, t, eps):
    """closed-form forward: x_t = sqrt(abar_t) x0 + sqrt(1-abar_t) eps -- one jump, no
chain"""
    return np.sqrt(abar[t])[:, None]*x0 + np.sqrt(1-abar[t])[:, None]*eps

X0, _ = make_moons(4000, noise=0.06, random_state=0); X0 = (X0 - X0.mean(0))/X0.std(0)

# eps-prediction MLP: input (x_t, sinusoidal t-embedding), output eps_hat
demb = 8
def temb(t):
    f = 10.0**np.arange(demb//2)
    a = t[:, None]/T*f
    return np.concatenate([np.sin(a), np.cos(a)], 1)
r = np.random.default_rng(0)
Din, Hd = 2+demb, 128
W1 = r.normal(0,(2/Din)**.5,(Din,Hd)); b1 = np.zeros(Hd)
W2 = r.normal(0,(2/Hd)**.5,(Hd,Hd));  b2 = np.zeros(Hd)
```

```
W3 = r.normal(0,(2/Hd)**.5,(Hd,2));   b3 = np.zeros(2)

def net(xt, t):
    inp = np.concatenate([xt, temb(t)], 1)
    h1 = np.maximum(0, inp@W1+b1); h2 = np.maximum(0, h1@W2+b2)
    return inp, h1, h2, h2@W3+b3

lr, B = 2e-3, 128
for it in range(20000):
    i = rng.integers(0, len(X0), B); t = rng.integers(0, T, B)
    eps = rng.normal(size=(B, 2)); xt = q_sample(X0[i], t, eps)
    inp, h1, h2, eh = net(xt, t)
    d = 2*(eh - eps)/B/2
    dW3 = h2.T@d; dh2 = (d@W3.T)*(h2 > 0)
    dW2 = h1.T@dh2; dh1 = (dh2@W2.T)*(h1 > 0)
    dW1 = inp.T@dh1
    for Pm, G in [(W3,dW3),(b3,d.sum(0)),(W2,dW2),(b2,dh2.sum(0)),(W1,dW1),
(b1,dh1.sum(0))]:
        Pm -= lr*G
ev = rng.normal(size=(512, 2)); tv = np.full(512, 50)
_,_,_,eh = net(q_sample(X0[:512], tv, ev), tv)
print(f"eps-prediction MSE at t=50: {((eh-ev)**2).mean():.3f}  (predict-zero baseline:
1.0)")

def sample(n):
    x = rng.normal(size=(n, 2))
    for t in range(T-1, -1, -1):
        tt = np.full(n, t)
        _,_,_,eh = net(x, tt)
        mean = (x - betas[t]/np.sqrt(1-abar[t])*eh)/np.sqrt(alphas[t])   # DDPM
posterior mean
        x = mean + (np.sqrt(betas[t])*rng.normal(size=(n, 2)) if t > 0 else 0)
    return x
Xs = sample(2000)
from scipy.spatial import cKDTree
d = cKDTree(X0).query(Xs)[0]
print(f"samples within 0.15 of the data manifold: {(d<0.15).mean():.3f}   median dist
{np.median(d):.4f}")

fig, axes = plt.subplots(1, 4, figsize=(11, 3))
for ax, t in zip(axes[:3], [0, 40, 99]):
    xt = q_sample(X0[:800], np.full(800, t), rng.normal(size=(800, 2)))
    ax.scatter(xt[:,0], xt[:,1], s=3, alpha=.4); ax.set(title=f'forward q(x_t),
t={t}', xlim=(-3,3), ylim=(-3,3))
axes[3].scatter(Xs[:,0], Xs[:,1], s=3, alpha=.4, c='tab:green');
axes[3].set(title='reverse samples', xlim=(-3,3), ylim=(-3,3))
for ax in axes: ax.set_aspect('equal')
fig.tight_layout(); fig.savefig("figures/ch18/fig_l5_ddpm.png", dpi=150)
print("figure saved: fig_l5_ddpm.png")
```

Output:

```
noise schedule: abar[0]=0.9999  abar[T-1]=0.009587  (signal all but gone by t=T)
eps-prediction MSE at t=50: 0.304  (predict-zero baseline: 1.0)
samples within 0.15 of the data manifold: 0.674   median dist 0.0684
figure saved: fig_l5_ddpm.png
```



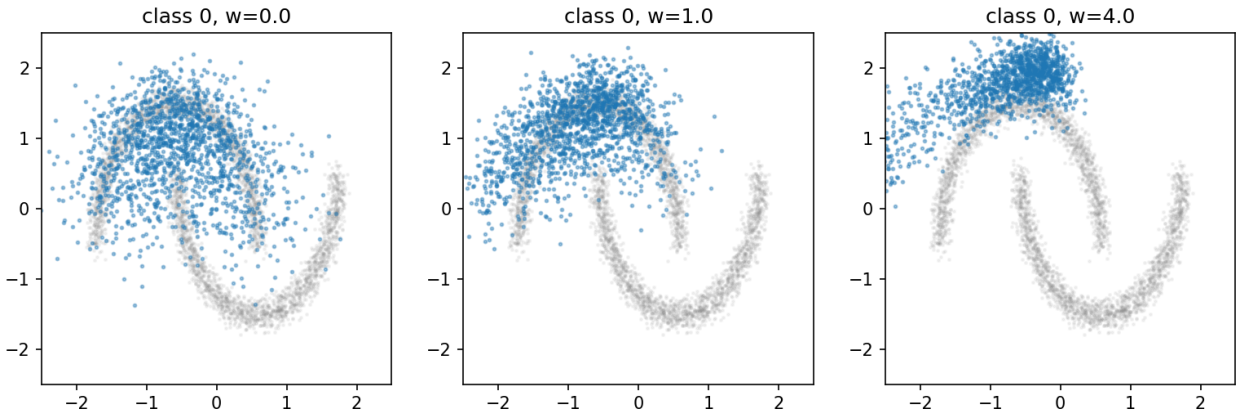
Listing 6 — Classifier-free guidance: one model, two conditions, and the w knob

```
"""Listing 6: classifier-free guidance -- one model, conditional and unconditional, and the w knob."""
import numpy as np
from sklearn.datasets import make_moons
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
rng = np.random.default_rng(6)

T = 100; betas = np.linspace(1e-4, 0.09, T); alphas = 1-betas; abar = np.cumprod(alphas)
X0, y0 = make_moons(4000, noise=0.06, random_state=0); X0 = (X0-X0.mean(0))/X0.std(0)

demb = 8
def temb(t):
    f = 10.0**np.arange(demb//2); a = t[:, None]/T*f
    return np.concatenate([np.sin(a), np.cos(a)], 1)
def cemb(c):                                          # class embedding: one-hot with a NULL class for CFG
    e = np.zeros((len(c), 3)); e[np.arange(len(c)), c] = 1; return e

r = np.random.default_rng(0)
Din, Hd = 2+demb+3, 128
W1 = r.normal(0,(2/Din)**.5,(Din,Hd)); b1 = np.zeros(Hd)
W2 = r.normal(0,(2/Hd)**.5,(Hd,Hd));  b2 = np.zeros(Hd)
W3 = r.normal(0,(2/Hd)**.5,(Hd,2));   b3 = np.zeros(2)
def net(xt, t, c):
    inp = np.concatenate([xt, temb(t), cemb(c)], 1)
    h1 = np.maximum(0, inp@W1+b1); h2 = np.maximum(0, h1@W2+b2)
    return inp, h1, h2, h2@W3+b3

lr, B = 2e-3, 128
for it in range(20000):
    i = rng.integers(0, len(X0), B); t = rng.integers(0, T, B)
    c = y0[i].copy(); c[rng.random(B) < 0.1] = 2
# DROP the label 10% of the time -> null class
    eps = rng.normal(size=(B,2))
    xt = np.sqrt(abar[t])[:,None]*X0[i] + np.sqrt(1-abar[t])[:,None]*eps
    inp, h1, h2, eh = net(xt, t, c)
    d = (eh - eps)/B
    dW3 = h2.T@d; dh2 = (d@W3.T)*(h2 > 0)
    dW2 = h1.T@dh2; dh1 = (dh2@W2.T)*(h1 > 0)
    for Pm, G in [(W3,dW3),(b3,d.sum(0)),(W2,dW2),(b2,dh2.sum(0)),(W1,inp.T@dh1),(b1,dh1.sum(0))]:
        Pm -= lr*G

def sample(n, cls, w):
    x = rng.normal(size=(n, 2)); cc = np.full(n, cls); cn = np.full(n, 2)
    for t in range(T-1, -1, -1):
        tt = np.full(n, t)
```

```
        ec = net(x, tt, cc)[3]; eu = net(x, tt, cn)[3]
        eh = (1+w)*ec - w*eu                                  # CFG: extrapolate cond away
from uncond
        mean = (x - betas[t]/np.sqrt(1-abar[t])*eh)/np.sqrt(alphas[t])
        x = mean + (np.sqrt(betas[t])*rng.normal(size=(n,2)) if t > 0 else 0)
    return x

from scipy.spatial import cKDTree
trees = [cKDTree(X0[y0 == c]) for c in range(2)]
print("guidance w   class-purity   on-manifold(<0.15)   spread(std of samples)")
fig, axes = plt.subplots(1, 3, figsize=(10.5, 3.4))
for ax, w in zip(axes, [0.0, 1.0, 4.0]):
    Xs = sample(1500, 0, w)
    d0 = trees[0].query(Xs)[0]; d1 = trees[1].query(Xs)[0]
    purity = (d0 < d1).mean(); onm = (d0 < 0.15).mean(); spread = Xs.std(0).mean()
    print(f"   {w:3.1f}         {purity:.3f}            {onm:.3f}
{spread:.3f}")
    ax.scatter(X0[:,0], X0[:,1], s=2, alpha=.08, c='gray')
    ax.scatter(Xs[:,0], Xs[:,1], s=3, alpha=.4)
    ax.set(title=f'class 0, w={w}', xlim=(-2.5,2.5), ylim=(-2.5,2.5));
ax.set_aspect('equal')
fig.tight_layout(); fig.savefig("figures/ch18/fig_l6_cfg.png", dpi=150)
print("figure saved: fig_l6_cfg.png")
```

Output:

```
guidance w   class-purity   on-manifold(<0.15)   spread(std of samples)
   0.0         0.799            0.593               0.677
   1.0         0.939            0.704               0.602
   4.0         1.000            0.361               0.823
figure saved: fig_l6_cfg.png
```



Listing 7 — A normalizing flow (RealNVP couplings): exact likelihood, exact inversion

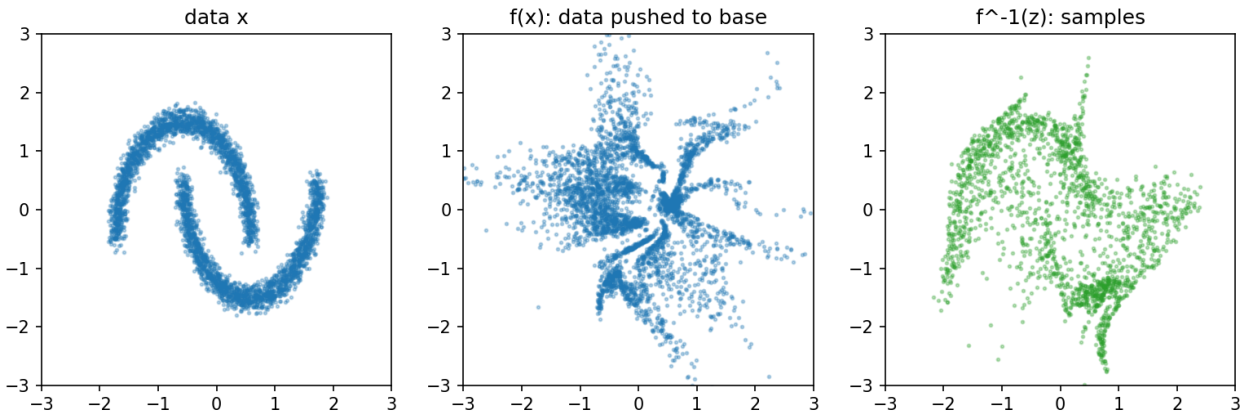
```
"""Listing 7: a normalizing flow (RealNVP couplings) -- exact likelihood, exact
inversion."""
import numpy as np
from sklearn.datasets import make_moons
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
rng = np.random.default_rng(7)

X0, _ = make_moons(4000, noise=0.06, random_state=0); X0 = (X0-X0.mean(0))/X0.std(0)
K, Hd = 6, 64                                          # 6 affine couplings, alternating
which coord passes through
```

```
def init(seed):
    r = np.random.default_rng(seed)
    return [dict(W1=r.normal(0,(2/1)**.5,(1,Hd)), b1=np.zeros(Hd),
                 Ws=r.normal(0,.01,(Hd,1)), bs=np.zeros(1),        # scale head (small
init: start near identity)
                 Wt=r.normal(0,.01,(Hd,1)), bt=np.zeros(1)) for _ in range(K)]
layers = init(0)

def coup_f(x, L, k):
    """x -> z: pass one coord, affine-transform the other with s,t nets of the passed
coord"""
    a, b = (x[:, :1], x[:, 1:]) if k % 2 == 0 else (x[:, 1:], x[:, :1])
    h = np.maximum(0, a@L['W1']+L['b1'])
    s = np.tanh(h@L['Ws']+L['bs']); t = h@L['Wt']+L['bt']
    bz = b*np.exp(s) + t
    z = np.concatenate([a, bz], 1) if k % 2 == 0 else np.concatenate([bz, a], 1)
    return z, s, (a, b, h, s, t)
def coup_inv(z, L, k):
    a, bz = (z[:, :1], z[:, 1:]) if k % 2 == 0 else (z[:, 1:], z[:, :1])
    h = np.maximum(0, a@L['W1']+L['b1'])
    s = np.tanh(h@L['Ws']+L['bs']); t = h@L['Wt']+L['bt']
    b = (bz - t)*np.exp(-s)
    return np.concatenate([a, b], 1) if k % 2 == 0 else np.concatenate([b, a], 1)

def fwd(x):
    z, logdet, caches = x, 0, []
    for k, L in enumerate(layers):
        z, s, c = coup_f(z, L, k); logdet = logdet + s.sum(1); caches.append(c)
    return z, logdet, caches

# exact invertibility BEFORE training (couplings are invertible by construction, not by
learning)
x = rng.normal(size=(500, 2)); z, _, _ = fwd(x)
for k in range(K-1, -1, -1): z = coup_inv(z, layers[k], k)
print(f"inverse(forward(x)) max error, untrained: {np.abs(z - x).max():.2e}")

lr, B = 1e-3, 256
for it in range(20000):
    xb = X0[rng.integers(0, len(X0), B)]
    z, logdet, caches = fwd(xb)
    # NLL = 0.5 z^2 - logdet ; backward: dz = z, and -1 into each layer's s.sum
    dz = z/B
    for k in range(K-1, -1, -1):
        L, (a, b, h, s, t) = layers[k], caches[k]
        da_pass, dbz = (dz[:, :1], dz[:, 1:]) if k % 2 == 0 else (dz[:, 1:], dz[:, :1])
        ds = dbz*b*np.exp(s) - (1/B)
# chain through bz = b e^s + t, and through -logdet
        dt = dbz
        db = dbz*np.exp(s)
        dsraw = ds*(1-s**2)                              # tanh
        dh = (dsraw@L['Ws'].T + dt@L['Wt'].T)*(h > 0)
        gW1 = a.T@dh; gWs = h.T@dsraw; gWt = h.T@dt
        da = dh@L['W1'].T + da_pass
        dz = np.concatenate([da, db], 1) if k % 2 == 0 else np.concatenate([db, da], 1)
        cl = lambda g: np.clip(g, -1, 1)
        L['W1'] -= lr*cl(gW1); L['b1'] -= lr*cl(dh.sum(0))
        L['Ws'] -= lr*cl(gWs); L['bs'] -= lr*cl(dsraw.sum(0))
        L['Wt'] -= lr*cl(gWt); L['bt'] -= lr*cl(dt.sum(0))
    if it % 10000 == 0:
        nll = (0.5*(z**2).sum(1) - logdet).mean() + np.log(2*np.pi)
        print(f"iter {it:5d}: exact NLL {nll:.3f} nats/point")
z, logdet, _ = fwd(X0)
print(f"final exact NLL on all data: {((0.5*(z**2).sum(1) - logdet).mean() +
np.log(2*np.pi)):.3f} nats")

Zs = rng.normal(size=(2000, 2)); Xs = Zs.copy()
for k in range(K-1, -1, -1): Xs = coup_inv(Xs, layers[k], k)
```

```
from scipy.spatial import cKDTree
d = cKDTree(X0).query(Xs)[0]
print(f"flow samples within 0.15 of manifold: {(d<0.15).mean():.3f}")

fig, axes = plt.subplots(1, 3, figsize=(10.5, 3.4))
axes[0].scatter(X0[:,0], X0[:,1], s=3, alpha=.3); axes[0].set_title('data x')
axes[1].scatter(*fwd(X0)[0].T, s=3, alpha=.3); axes[1].set_title('f(x): data pushed to
base')
axes[2].scatter(Xs[:,0], Xs[:,1], s=3, alpha=.3, c='tab:green');
axes[2].set_title('f^-1(z): samples')
for ax in axes: ax.set_aspect('equal'); ax.set(xlim=(-3,3), ylim=(-3,3))
fig.tight_layout(); fig.savefig("figures/ch18/fig_l7_flow.png", dpi=150)
print("figure saved: fig_l7_flow.png")
```

Output:

```
inverse(forward(x)) max error, untrained: 1.33e-15
iter     0: exact NLL 2.803 nats/point
iter 10000: exact NLL 1.866 nats/point
final exact NLL on all data: 2.136 nats
flow samples within 0.15 of manifold: 0.786
figure saved: fig_l7_flow.png
```



Listing 8 — FID and Inception Score from scratch: what the numbers measure

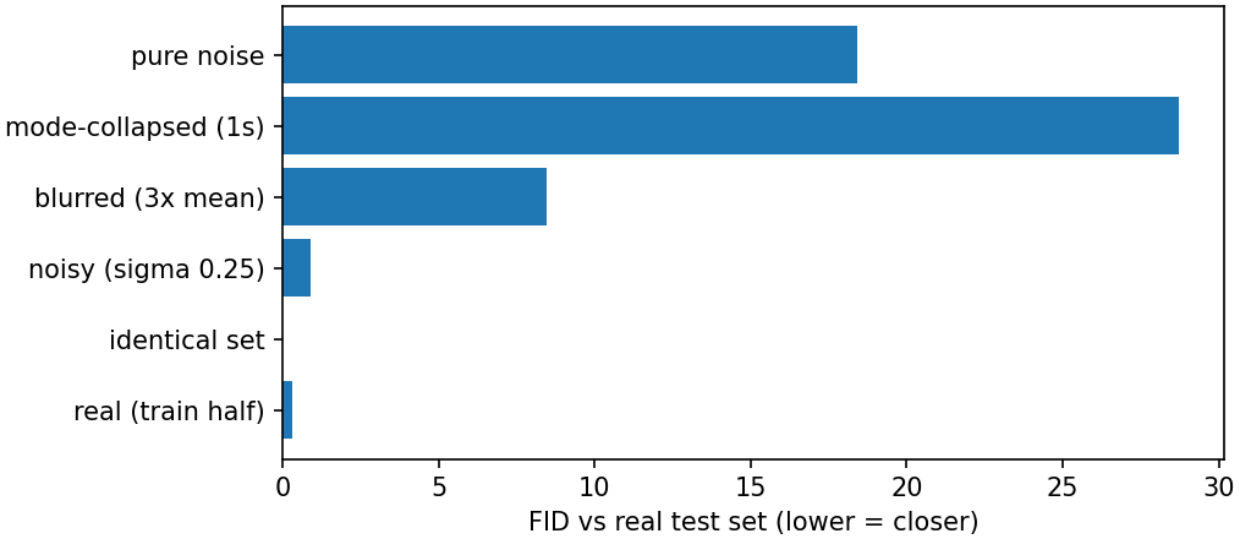
```
"""Listing 8: FID and Inception Score from scratch -- what the numbers actually
measure."""
import numpy as np
from sklearn.datasets import load_digits
from scipy.linalg import sqrtm
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
rng = np.random.default_rng(8)

X, y = load_digits(return_X_y=True); X = X/16.0
idx = rng.permutation(len(X)); tr, te = idx[:1200], idx[1200:]

# stand-in "Inception": a small classifier; features = penultimate layer, p(y|x) = its
softmax
r = np.random.default_rng(0)
W1 = r.normal(0,(2/64)**.5,(64,32)); b1 = np.zeros(32)
W2 = r.normal(0,(2/32)**.5,(32,10)); b2 = np.zeros(10)
for it in range(30000):
    i = tr[rng.integers(len(tr))]
    h = np.maximum(0, X[i]@W1+b1); zl = h@W2+b2
```

```
    e = np.exp(zl-zl.max()); p = e/e.sum()
    d = p.copy(); d[y[i]] -= 1
    W2 -= .01*np.outer(h, d); b2 -= .01*d
    dh = (W2@d)*(h > 0); W1 -= .01*np.outer(X[i], dh); b1 -= .01*dh
feat = lambda Xs: np.maximum(0, Xs@W1+b1)
def probs(Xs):
    zl = feat(Xs)@W2+b2; e = np.exp(zl-zl.max(1, keepdims=True)); return e/e.sum(1,
keepdims=True)
acc = (probs(X[te]).argmax(1) == y[te]).mean()
print(f"feature-extractor classifier accuracy: {acc:.3f}")

def fid(A, B):
    fa, fb = feat(A), feat(B)
    mua, mub = fa.mean(0), fb.mean(0)
    eye = np.eye(fa.shape[1])*1e-6                      # numerical jitter: keep the
product PSD
    Ca, Cb = np.cov(fa.T)+eye, np.cov(fb.T)+eye
    covmean = sqrtm(Ca@Cb).real
    return ((mua-mub)**2).sum() + np.trace(Ca + Cb - 2*covmean)
def inception_score(A):
    p = probs(A); py = p.mean(0)
    kl = (p*(np.log(p+1e-12) - np.log(py+1e-12))).sum(1)
    return np.exp(kl.mean())

real = X[te]
sets = {
    "real (train half)":  X[tr[:600]],
    "identical set":      real.copy(),
    "noisy (sigma 0.25)": np.clip(real + rng.normal(0,.25,real.shape), 0, 1),
    "blurred (3x mean)":  np.array([np.convolve(im, np.ones(3)/3, 'same') for im in
real]),
    "mode-collapsed (1s)": X[tr][y[tr] == 1][:150].repeat(4, 0),
    "pure noise":         rng.uniform(0, 1, real.shape),
}
print(f"\n{'candidate set':22s}  {'FID':>8s}  {'IS':>6s}")
for name, S in sets.items():
    print(f"{name:22s}  {fid(real, S):8.2f}  {inception_score(S):6.2f}")
print(f"(real data IS: {inception_score(real):.2f}; IS ceiling = number of classes =
10)")
print("note: mode collapse -> LOW IS (p(y) collapses onto p(y|x)) and HIGH FID
(covariance shrinks);")
print("      but IS never compares to real data -- a memorized single real image per
class scores well")

names = list(sets); fids = [fid(real, sets[k]) for k in names]
fig, ax = plt.subplots(figsize=(7, 3.2))
ax.barh(names, fids); ax.set_xlabel('FID vs real test set (lower = closer)')
fig.tight_layout(); fig.savefig("figures/ch18/fig_l8_fid.png", dpi=150)
print("figure saved: fig_l8_fid.png")
```

Output:

```
feature-extractor classifier accuracy: 0.973

candidate set                FID      IS
real (train half)           0.31    9.19
identical set              -0.00    9.05
noisy (sigma 0.25)          0.89    7.90
blurred (3x mean)           8.44    6.48
mode-collapsed (1s)        28.74    1.13
pure noise                 18.41    3.26
(real data IS: 9.05; IS ceiling = number of classes = 10)
note: mode collapse -> LOW IS (p(y) collapses onto p(y|x)) and HIGH FID (covariance
shrinks);
      but IS never compares to real data -- a memorized single real image per class
```

```
scores well
figure saved: fig_l8_fid.png
```



Interview questions and answers

Q1. Why can't you sample from a plain autoencoder? Be precise about what's missing.

Sampling needs a known distribution over latents to draw from; the AE's objective (reconstruction only) says NOTHING about where codes live. Listing 1's latent space: cluster mean (-2.6, -0.1), uneven scales, voids between digit clusters — 34% of N(0,1) draws land >1.0 from any encoded training point, and the decoder on such coordinates emits smeared non-digits. Two fixes exist: constrain the latent to match a prior during training (VAE — KL term), or fit a density to the encoded latents afterward (an ex-post prior). Without one of them, an AE is a compressor, not a generator.

Q2. Derive the ELBO in three steps. What is the gap between the bound and the true likelihood?

(1) log p(x) = log ∫ p(x|z)p(z)dz. (2) Multiply and divide by q(z|x), apply Jensen: log E_q[p(x|z)p(z)/q] >= E_q[log p(x|z)] - KL(q(z|x)||p(z)). (3) The two terms: expected reconstruction under the approximate posterior, minus posterior-to-prior KL. Exact identity: log p(x) = ELBO + KL(q(z|x)||p(z|x)) — the gap IS the posterior-approximation error, so improving q tightens the bound without touching the model. For Gaussian q and prior the KL is closed-form: half of sum(sigma^2 + mu^2 - 1 - log sigma^2) (Listing 2 implements it and its gradients by hand).

Q3. Explain the reparameterization trick and why the gradient can't just flow through a sample.

"Backprop through z ~ N(mu, sigma)" is undefined — sampling is not a differentiable op, and the DISTRIBUTION depends on the parameters you need gradients for. Rewrite z = mu + sigma·eps with eps ~ N(0,1): randomness now enters as an INPUT, z is a deterministic differentiable function of (mu, sigma), and d z/d mu = 1, d z/d sigma = eps. Listing 2's check freezes eps and central-differences straight through the sampled path: 5.2e-9. Requirements: the density must admit such a location-scale (or more general invertible-CDF) form and the downstream function must be differentiable — Gaussians, Laplace yes; discrete latents no (use REINFORCE or Gumbel-softmax).

Q4. Pathwise vs score-function gradient estimators: trade-offs and measured variance.

Score function/REINFORCE: grad E[f] = E[f(z)·grad log p(z)] — unbiased, needs only the density's differentiability, works for discrete z and black-box f; variance scales with f's magnitude. Pathwise/reparameterized: grad E[f] = E[f'(z)·dz/dtheta] — needs differentiable f and reparameterizable p, but uses f's SLOPE, not its value. Listing 3 on E[z^2], true grad 6: pathwise variance 4.0, REINFORCE 220 (55x), baseline-corrected REINFORCE 81. Rule: reparameterize whenever possible (VAEs); REINFORCE when structurally forced (RL rewards, discrete latents), and then spend effort on baselines/control variates.

Q5. What is posterior collapse, when does it happen, and what helps?

The KL term is minimized by q(z|x) = p(z) for all x — latent carries zero information. If the decoder is powerful enough to model x WITHOUT z (autoregressive decoders are the classic case), the optimizer happily takes that route: reconstruction stays good, KL -> 0, z is dead — generation still works but is uncontrollable, and representations are useless. Detection: per-dimension KL near zero; mutual information estimates. Remedies: KL annealing (warm up the KL weight from 0), free bits (only penalize KL above a floor per dimension), weaker/restricted decoders, skip connections from z to every decoder layer, or delta-VAE constraints. Listing 2's KL of 1.6 nats is low but nonzero — the 2-D latent still carries the class structure visible in its figure.

Q6. Why are VAE samples blurry?

Three compounding reasons. (1) The likelihood: pixelwise Gaussian/Bernoulli p(x|z) makes the optimal decoder output the MEAN of all x consistent with z — averaging sharp alternatives is blur. (2) The bound: optimizing the ELBO, not the likelihood, biases q toward over-dispersed posteriors; decoders average over that uncertainty. (3) Prior holes: mismatch between the aggregate posterior and N(0,1) means some prior draws decode from thinly-trained regions. Fixes in the literature: richer likelihoods (discretized logistics), adversarial or perceptual reconstruction losses (VAE-GAN), hierarchical latents (NVAE), or using the VAE only as a compressor with a stronger generator in latent space — which is literally the Stable Diffusion architecture.

Q7. Write the GAN minimax objective and explain what D and G each see at the optimum.

min_G max_D E_x[log D(x)] + E_z[log(1-D(G(z)))]. For fixed G the optimal discriminator is D*(x) = p_data(x)/(p_data(x)+p_g(x)) — a density-ratio estimator, which is the deep fact: D learns WHERE the generator's distribution deviates. Substituting D* reduces G's objective to 2·JSD(p_data||p_g) - log 4: at the game's equilibrium p_g = p_data and D* = 1/2 everywhere. None of this survives contact with SGD on non-convex networks — training is a simultaneous two-player gradient dance with no guaranteed convergence, hence the stabilization industry (Q9).

Q8. Why does the original minimax G-loss fail in practice, and what is the standard fix?

Early training: G is bad, D wins, D(G(z)) ~ 0. The minimax G-gradient through log(1-D) is proportional to 1/(1-D) ~ 1 — FLAT precisely when G most needs signal. The non-saturating loss -log D(G(z)) has gradient proportional to 1/D ~ 50 at D(fake)=0.02 (Listing 4's table: 50x). Same fixed point, opposite gradient behavior at the start. Measured consequence on the 8-Gaussian ring: minimax 0/8 modes covered after 20k iterations; non-saturating 7/8. This is the first thing Goodfellow's own paper changed about its own objective, and interviewers love that detail.

Q9. Explain mode collapse: mechanism, detection, and remedies.

Mechanism: G's objective pays for fooling D, not for covering p_data — concentrating on whichever modes currently fool D is a legitimate strategy; D then adapts, G hops modes, or settles on a subset. Maximum likelihood punishes missing modes infinitely (zero mass on real data = infinite NLL); the GAN objective doesn't — the asymmetry is the root cause. Listing 4's non-saturating GAN: 7/8 modes with counts 2-74 — partial collapse on camera. Detection: FID's covariance term (Listing 8: collapsed set FID 28.7), low IS (1.13), birthday-paradox duplicate tests, per-mode counts when modes are known. Remedies: WGAN/WGAN-GP (non-saturating divergence), minibatch discrimination (D sees batch statistics), unrolled D, packing, diversity terms — none fully solved; diffusion sidestepped it by returning to likelihood-family objectives.

Q10. What did WGAN change, exactly, and why does it help?

Replace JS divergence with the Wasserstein-1 (Earth-Mover) distance via its Kantorovich-Rubinstein dual: max over 1-Lipschitz critics f of E_data[f(x)] - E_g[f(x)]. Why it helps: when supports of p_data and p_g barely overlap (the typical high-dimensional case), JS saturates to log 2 — zero gradient — while W1 varies smoothly with how FAR the distributions are, so the critic's gradient keeps pointing somewhere useful even when it classifies perfectly. Enforcement of Lipschitz-ness is the implementation question: weight clipping (original, crude), gradient penalty on interpolates (WGAN-GP, standard), or spectral normalization (constrain each layer's largest singular value). Bonus: the critic's loss correlates with sample quality — a usable training curve, which vanilla GANs lack.

Q11. Walk through DDPM's forward process and derive why one-jump corruption is possible.

q(x_t|x_(t-1)) = N(sqrt(1-beta_t)·x_(t-1), beta_t·I): shrink toward zero, add noise. Composing Gaussians stays Gaussian — recursing gives q(x_t|x_0) = N(sqrt(abar_t)·x_0, (1-abar_t)·I) with abar_t = prod(1-beta_s): the scale factors multiply, the variances accumulate to exactly 1-abar_t (variance-preserving: total variance stays 1 for unit-variance data). So x_t = sqrt(abar_t)·x_0 + sqrt(1-abar_t)·eps in ONE step — training can sample any t uniformly without simulating the chain (Listing 5). Schedule design = making abar_T ~ 0 (the listing: 0.0096) so x_T is indistinguishable from the N(0,I) that sampling starts from; a schedule that leaves signal at T (the listing's first attempt left 8%) makes sampling start from the wrong distribution.

Q12. Why does DDPM predict the NOISE rather than the clean image, and what is the connection to score matching?

The ELBO of the diffusion hierarchy reduces (Ho et al.'s simplification, dropping per-term weights) to ||eps - eps_theta(x_t, t)||^2 — and eps-prediction empirically beats x0-prediction: the target is unit-scale at every t (x0-prediction must span wildly different signal levels), and the weighting it implies emphasizes the mid-noise steps where structure forms. The score connection: from x_t = sqrt(abar)x0 + sqrt(1-abar)eps, the score of the marginal is grad log p(x_t) = -eps/sqrt(1-abar) — predicting eps IS estimating the score, sampling is discretized Langevin/reverse-SDE, and the whole model family (DDPM, score-based SDE, v-prediction) are parameterizations of one object. Modern refinement worth naming: v-prediction (a mix of eps and x0) for numerical stability at high noise.

Q13. Explain classifier-free guidance, including the training trick and the failure at high w.

Train ONE network on both conditional and unconditional denoising by dropping the label to a null token ~10% of the time (Listing 6). Sample with the extrapolation eps_hat = (1+w)·eps(x|c) - w·eps(x) — moving PAST the conditional prediction, away from the unconditional one; in score terms it sharpens p(c|x)^w, upweighting regions where the condition is most identifiable. Measured sweep (Listing 6): purity 0.799/0.939/1.000 at w=0/1/4 — but on-manifold fidelity peaks at w=1 (0.704) and collapses at w=4 (0.361): over-guidance pushes samples off the data manifold — the 2-D version of fried, over-saturated high-guidance images. Predecessor: classifier guidance (gradients of an external classifier on noisy images) — CFG won because it needs no separate classifier and guides with the generator's own knowledge.

Q14. What is latent diffusion, and why did it make text-to-image practical?

Run diffusion in the latent space of a pretrained autoencoder (KL- or VQ-regularized), not in pixels: a 512x512x3 image becomes 64x64x4 latents — ~48x fewer dimensions, so each of the hundreds of denoising steps costs ~64x less, and the U-Net's capacity is spent on semantics rather than imperceptible high-frequency detail (which the decoder handles once at the end). Conditioning: text through a frozen encoder (CLIP-family), injected via cross-attention at each U-Net block (Chapter 16's mechanism verbatim). That stack — VAE compressor + latent U-Net denoiser + CLIP text conditioning + CFG at sampling — IS Stable Diffusion. The design lesson interviews probe: put the expensive iterative model where dimensions are few, the cheap feedforward models where they are many.

Q15. Compare sampling cost across the four families, and name the diffusion speedups.

GAN: one forward pass — real-time. VAE: one decoder pass. Flow: one inverse pass (couplings invert algebraically). Diffusion: one network pass PER STEP, classically 1000 — three orders slower. Speedups: DDIM (deterministic reverse process, subsample to ~50 steps, same trained network); better solvers (DPM-Solver, ~10-20 steps); distillation (progressive: student halves steps repeatedly; consistency models: 1-4 steps); latent diffusion (cheaper per step, Q14). The trade is fundamental: diffusion buys its quality by spending compute at inference, and the entire post-2021 research program is buying that quality back cheaper.

Q16. Write the change-of-variables formula and explain the affine coupling's two design wins.

log p(x) = log p_z(f(x)) + log|det J_f(x)| — exact likelihood for invertible f. Naive det costs O(d^3); coupling makes it free: split x into (a, b); output (a, b·exp(s(a)) + t(a)). Win 1: the Jacobian is triangular — det = exp(sum s), log-det = sum of the scale outputs, O(d). Win 2: inversion is algebraic — b = (b' - t(a))·exp(-s(a)) — s and t are NEVER inverted, so they can be arbitrary networks. Listing 7: inverse(forward(x)) = 1.3e-15 with UNTRAINED networks (invertibility is architectural), exact NLL trained 2.80 -> 1.87 nats. Alternate masks/permutations between couplings so every dimension gets transformed; Glow's 1x1 convs are learned permutations.

Q17. Why do flows preserve dimension, and what does that cost?

Invertibility forces a bijection: latent dimension = data dimension, no bottleneck, no compression. Costs: (1) parameters and compute scale with data dimension — high-res images need very deep stacks (Glow: ~600 couplings-equivalent) for modest quality; (2) the model must assign density to EVERY pixel detail, spending capacity on imperceptible high frequencies (same pathology latent diffusion escapes, Q14); (3) topology: a continuous bijection cannot map a unimodal Gaussian onto disconnected modes exactly — density stretches thin filaments between modes (visible if you zoom Listing 7's samples between the moons). Uses where flows still win: exact density for anomaly detection/physics, invertible feature extractors, and as glue components inside larger models.

Q18. Define Inception Score and give the memorization argument against it.

IS = exp(E_x[KL(p(y|x) || p(y))]) over generated samples through a fixed classifier: high when each sample is confidently classifiable (sharp p(y|x)) AND the class marginal is broad (uniform p(y)) — ceiling = class count (Listing 8: real digits 9.05/10). The killer flaw: IS never touches real data. A generator that memorizes ONE flawless image per class achieves near-ceiling IS with zero diversity within classes and zero novelty; IS also ignores within-class fidelity entirely and inherits the classifier's domain (ImageNet IS on faces is meaningless). Listing 8's mode-collapsed set: IS 1.13 — it does catch CLASS-level collapse, just nothing finer.

Q19. Write the FID formula and explain what each term catches. What are its blind spots?

FID = ||mu_r - mu_g||^2 + Tr(C_r + C_g - 2(C_r·C_g)^(1/2)) — Frechet distance between Gaussians fitted to real and generated features (Inception pool3 in practice; Listing 8 uses a digits classifier's penultimate layer). Mean term: are you in the right REGION of feature space. Covariance term: do you have the right SPREAD — this is what catches mode collapse (Listing 8: all-ones set FID 28.7 while noisy real data scores 0.89). Blind spots: Gaussian assumption on features; sample-size bias (fresh real-vs-real scored 0.31, not 0 — always compare at matched n); feature-extractor bias (ImageNet-trained = texture-heavy); silent about memorization (nearest-neighbor audits) and about WHICH failure (precision/recall decompositions separate quality from coverage).

Q20. Sanity checks before trusting a reported FID improvement of 0.5?

(1) Same feature extractor and implementation — FID varies across Inception weights/preprocessing (resize/crop) by more than 0.5. (2) Same sample counts both sides — bias shrinks with n (Listing 8's real-vs-real floor of 0.31 at n=600 would shrink with more samples); 50k is conventional. (3) Multiple seeds with variance — one draw of 50k has noise. (4) Same guidance/truncation settings — CFG weight changes FID dramatically (Listing 6's fidelity sweep), so a "model" improvement may be a knob change. (5) Check test-set contamination and memorization (nearest neighbors in feature space). A 0.5 delta that survives all five is real; most don't.

Q21. Place VAE, GAN, flow, diffusion on the quality / speed / likelihood triangle with one-line justifications.

VAE: fast sampling + (bounded) likelihood, weakest raw sample quality — pixel-likelihood blur (Q6). GAN: fast + sharp, NO likelihood — density ratio learned implicitly, mode coverage unguaranteed (Q9). Flow: exact likelihood + one-pass sampling, quality limited by invertibility handcuffs (Q17). Diffusion: top quality + likelihood (as a bound, tighter than VAE's in practice), sampling hundreds of passes (Q15). Deployment mapping: real-time interactive -> GAN/distilled diffusion; density/anomaly work -> flows; representation + generation compromise -> VAE; quality-first offline generation -> diffusion. Modern systems compose them: Stable Diffusion = VAE + diffusion + CLIP.

Q22. Your diffusion model trained on medical scans produces beautiful samples; a clinician says some contain plausible-looking pathologies that never co-occur anatomically. Diagnose and respond.

Diagnosis: the model learned per-region marginals well but long-range joint structure imperfectly; FID won't catch it (local feature statistics fine — the same blindness as its texture bias). Also audit for memorization (patient-privacy hazard: nearest-neighbor search in feature space between samples and training scans). Responses: condition on anatomy (segmentation maps/templates) so global structure is supplied, not sampled; evaluate with domain-specific consistency checks (rule-based anatomical validators, downstream-task performance of models trained on synthetic data); reduce guidance weight if over-guidance is pushing off-manifold combinations (Listing 6's w=4 regime). For synthetic-data-for-training use cases, report utility metrics (train-on-synthetic test-on-real), not just fidelity metrics.

Q23. How does StyleGAN's design differ from DCGAN's, and what does each piece buy?

DCGAN: z feeds the first layer, all-conv generator, BN — the stable-recipe baseline. StyleGAN: (1) a mapping network z -> w unwarps the prior into a learned style space W whose directions disentangle attributes (pose, age) far better than z-space; (2) w injects at EVERY resolution via AdaIN/modulation — coarse layers control pose/geometry, fine layers texture — enabling style mixing across scales; (3) per-pixel noise inputs supply stochastic detail (hair, pores) so w carries only structure; (4) progressive/carefully-regularized training (path-length penalty, R1). Interview one-liner: DCGAN made GANs train; StyleGAN made their latent spaces steerable — and W-space editing (GAN inversion) became its own subfield.

Q24. Derive the closed-form KL between N(mu, sigma^2) and N(0,1) used in the VAE loss.

KL = E_q[log q - log p] with q = N(mu, sigma^2), p = N(0,1). log q(z) = -0.5 log(2 pi sigma^2) - (z-mu)^2/(2 sigma^2); log p(z) = -0.5 log(2 pi) - z^2/2. Take E_q: E[(z-mu)^2] = sigma^2 kills the first quadratic to -1/2; E[z^2] = sigma^2 + mu^2. Sum: KL = 0.5(sigma^2 + mu^2 - 1 - log sigma^2), summed over dimensions (Listing 2's line). Sanity: zero iff mu=0, sigma=1; quadratic in mu (mean displacement is cheap); log-barrier as sigma -> 0 (collapsing the posterior to a point is expensive — that barrier is what keeps q from becoming deterministic).

Q25. Design a generative system for product-photo backgrounds: input product cutout, output varied photorealistic scenes, 500ms budget, brand-safety constraints.

Backbone: latent diffusion inpainting — freeze the product region, denoise only the background, conditioned on text prompts (scene descriptors) via cross-attention; the product mask enters as U-Net input channels. 500ms budget -> distilled few-step sampler (consistency/LCM-style, 2-4 steps) in latent space; VAE decode once. Brand safety: curated prompt templates rather than free text; CFG weight tuned on the fidelity/diversity sweep (Listing 6's logic); NSFW/logo filters on outputs; and a nearest-neighbor check against licensed training data if provenance matters. Evaluate: FID against a held-out set of real product shots at matched n (Q20), human preference on a rubric, and product-edge artifacts specifically (compositing seams are the known failure). Fallback tier: template compositing when the sampler's confidence proxies fail.

Q26. Both VAEs and diffusion models maximize an ELBO. What is actually different?

Structure of the latent hierarchy and what is learned. VAE: ONE stochastic layer (or few), BOTH directions learned — encoder q(z|x) and decoder p(x|z) trained jointly, and the bound's looseness (posterior gap) plus pixel likelihood produce blur. Diffusion: ~1000 stochastic layers with the FORWARD direction fixed by design (Gaussian corruption, no parameters) — only the reverse is learned, each step a small, nearly-Gaussian denoising that a network can fit well, so the composite bound is tight in practice. Fixing the inference path removes the posterior-collapse failure mode entirely (nothing to collapse) at the price of Q15's sampling cost. Same objective family, opposite ends of the depth-of-hierarchy dial.

Q27. When would you still choose a GAN in the diffusion era?

(1) Latency: single-pass generation for real-time applications — interactive editing, streaming, game assets; distilled diffusion narrows but rarely beats a tuned GAN at equal quality-per-millisecond in narrow domains. (2) Narrow domains with abundant data (faces): StyleGAN-class quality is competitive and its W-space editing is more controllable than diffusion inversion. (3) Compact deployment: one small generator vs a U-Net run repeatedly. (4) As components: adversarial losses inside other systems (VAE-GAN, super-resolution perceptual losses, diffusion-GAN hybrids for few-step sampling). The honest framing: adversarial training became an ingredient rather than a flagship — likelihood-family objectives won the frontier because they scale predictably.

Q28. This chapter's measured claims in one sweep.

AE prior broken: 34% of N(0,1) draws in latent holes (Listing 1). VAE: gradient THROUGH the sample checked 5.2e-9; aggregate posterior pinned to ~N(0,1); prior samples decode (Listing 2). Reparameterization: variance 4.0 vs REINFORCE 220 — 55x (Listing 3). GAN: minimax gradient 50x too weak when D wins -> 0/8 modes; non-saturating 7/8 with counts 2-74 = partial collapse (Listing 4). DDPM: abar_T driven to 0.0096, eps-MSE 0.30 vs 1.0 baseline, two-moons from noise at median manifold distance 0.068 (Listing 5). CFG: purity 0.80 -> 1.00 while fidelity peaks at w=1 then halves at w=4 (Listing 6). Flow: inversion exact at 1.3e-15 before training; exact NLL 2.80 -> 1.87 (Listing 7). Metrics: FID 0 on identical sets, 0.31 sampling floor, 28.7 on mode collapse (covariance term); IS 9.05 real, 1.13 collapsed, and IS never sees real data (Listing 8).